Estudio comparativo del rendimiento de un árbol de decisión C4.5 propio y del modelo de ensamble Random Forest en problemas de clasificación

Laura Cornaló Bassan[1 [00000]], Matías Fernandez[1 [24282]], Mariano Insaurralde[1 [24684]], Philippe Maurel[1 [25613]], y Franco Sanchez[1 [27310]]

Universidad Tecnológica Nacional, Facultad Regional de Resistencia
Ingeniería en Sistemas de Información
{correo1,correo2,correo3,correo4,correo5}@gmail.com

Resumen Este trabajo compara una implementación propia del árbol de decisión C4.5 con el modelo de ensamble Random Forest sobre el conjunto de datos Heart Disease, con 10 particiones estratificadas. Con los valores por defecto, Random Forest alcanzó una tasa de acierto de 0,811 y C4.5, de 0,718. Limitar el crecimiento del árbol elevó su tasa hasta 0,757. Random Forest se mostró poco sensible a sus parámetros, mientras que C4.5 ofreció un modelo legible a cambio de requerir ajuste.

Palabras clave: Árbol de Decisión · C4.5 · Random Forest

1 Introducción

Un clasificador automático asigna una clase a cada caso a partir de sus características. Los árboles de decisión producen un modelo legible, en el que cada camino de la raíz a una hoja equivale a una regla [3]. Random Forest combina muchos árboles para reducir la dependencia de una sola estructura, a cambio de más cómputo y de una decisión que ya no se sigue en un único camino [1]. C4.5 elige cada pregunta con un criterio orientado a separar las clases y admite atributos numéricos y categóricos [3]. Ninguno de los dos enfoques domina al otro en general: la elección depende del problema, de la métrica que se priorice y del costo que se acepte pagar.

Este trabajo compara una implementación propia de C4.5 con el Random Forest de scikit-learn [6] sobre el conjunto de datos Heart Disease, en su versión de Cleveland [2]. La comparación usa las mismas particiones, las mismas métricas y las mismas condiciones de evaluación. Con esquemas de evaluación distintos, una diferencia de rendimiento puede reflejar la forma de medir y no el algoritmo. El objetivo es determinar en qué condiciones conviene cada uno, atendiendo a la tasa de acierto (proporción de predicciones correctas), la estabilidad de las predicciones ante cambios de semilla, el tiempo de cómputo y la interpretabilidad.

Las contribuciones son una implementación propia de C4.5 con decisiones explícitas sobre la construcción, el criterio de división y el control del crecimiento; un protocolo común de evaluación; y un análisis comparativo de esos cuatro aspectos.

El documento describe primero los fundamentos de ambos modelos, luego la implementación, después los experimentos y cierra con las conclusiones.

2 Fundamentos teóricos

2.1 Árboles de decisión y C4.5

C4.5 construye un árbol de decisión de forma recursiva [3]. En cada nodo elige la pregunta sobre un atributo que mejor separa las clases y repite el procedimiento en los subconjuntos resultantes. La entropía mide la impureza de un nodo a partir de la distribución de sus clases. Si D es el conjunto de casos, C el conjunto de clases y p_c la proporción de casos de la clase c, medida por masa cuando los pesos no son unitarios, la entropía se define en la ecuación (1):

$$H(D) = -\sum_{c \in C} p_c \log_2 p_c. \tag{1}$$

La entropía vale cero cuando todos los casos pertenecen a una misma clase y alcanza su máximo cuando las clases están equilibradas.

Una pregunta sobre un atributo A divide D en v subconjuntos D_j, con masas W_j, y W es la masa de D. La ganancia de información es la reducción de entropía que produce esa división, ecuación (2), donde la suma ponderada representa la impureza que queda en las ramas. Cuanto mayor es la ganancia, más puras quedan las ramas, y una ganancia nula indica que la partición no reduce la impureza.

$$\mathrm{Gain}(D, A) = H(D) - \sum_{j=1}^{v} \frac{W_j}{W} H(D_j). \tag{2}$$

La ganancia favorece a los atributos con muchos valores distintos, porque fragmentar el conjunto reduce la impureza casi por construcción. C4.5 atenúa ese efecto con la información de la partición, ecuación (3), que mide cuánto fragmenta el atributo. La tasa de ganancia, ecuación (4), es el cociente entre ambas magnitudes; un atributo que fragmenta demasiado queda penalizado. Si la partición no produce al menos dos ramas conocidas de masa positiva, el atributo no se considera candidato.

$$\mathrm{SplitInfo}(D, A) = -\sum_{j=1}^{v} \frac{W_j}{W} \log_2 \frac{W_j}{W}, \tag{3}$$

$$\mathrm{GainRatio}(D, A) = \frac{\mathrm{Gain}(D, A)}{\mathrm{SplitInfo}(D, A)}. \tag{4}$$

Los atributos numéricos requieren un tratamiento distinto. En cada nodo se ordenan los valores observados, operación de costo $O(|D| \log |D|)$, y se examinan

los $r-1$ cortes entre valores consecutivos; cada corte separa los casos con $A \leq t$ de los que tienen $A > t$. Se conserva el corte con la mayor ganancia, porque con la tasa de ganancia prevalecerían los cortes que aíslan muy pocos casos [4]. El umbral es un valor observado y se elige en cada nodo, sin discretización global. Un atributo numérico puede reutilizarse en niveles inferiores, mientras que un atributo categórico se descarta después de usarse [3].

Con valores faltantes, sea W_K la masa de los casos con valor conocido para el atributo y W_U la de los que no lo tienen. La ganancia se calcula solo sobre el subconjunto D_K de casos conocidos y se escala por la fracción W_K/W, ecuación (5). La información de la partición agrega la masa faltante W_U como una rama de cálculo, aunque no origine una rama del árbol. La tasa de ganancia ajustada es el cociente entre la ganancia ajustada y esa información de la partición. En el entrenamiento, un caso faltante se envía a todas las ramas y su peso se reparte según $q_j = W_j/W_K$, ecuación (6); en la predicción, las probabilidades de las ramas se combinan con esas mismas proporciones [3].

$$\text{Gain}^*(D, A) = \frac{W_K}{W}\left[H(D_K) - \sum_j \frac{W_j}{W_K} H(D_j)\right], \tag{5}$$

$$q_j = \frac{W_j}{W_K}. \tag{6}$$

Este mecanismo es propio de C4.5.

Un árbol que crece sin restricciones puede ajustarse a rasgos particulares del conjunto de entrenamiento y perder capacidad sobre casos nuevos. La prepoda detiene el crecimiento durante la construcción; la pospoda elimina subárboles después de construir el árbol completo. El C4.5 original emplea pospoda pesimista, mientras que este trabajo aplica solo prepoda y no implementa poda de reglas [3]. La construcción es codiciosa: cada división se elige con la información disponible en el nodo y no se revisa después, de modo que la prepoda puede detener el crecimiento antes de lo conveniente.

2.2 Random Forest

Random Forest combina B árboles de decisión construidos con perturbaciones aleatorias [1]. Cada árbol se entrena con una muestra del mismo tamaño que el conjunto original, obtenida con reemplazo, y en cada división evalúa solo un subconjunto aleatorio de los atributos disponibles. Los árboles crecen sin poda y la predicción combina sus respuestas por voto mayoritario o por promedio de probabilidades. Esa doble fuente de aleatoriedad reduce la dependencia de una sola estructura y la correlación entre los árboles.

Breiman muestra que el error de generalización depende de la fuerza de los árboles y de su correlación: árboles fuertes y poco correlacionados tienden a reducir el error [1]. La fuerza resume la separación media entre la clase correcta y las alternativas, y la correlación mide cuánto se parecen las respuestas de dos árboles. El análisis de este trabajo es empírico y no usa esa caracterización para

calcular el error. En este trabajo se usa la implementación de scikit-learn (...); su configuración se detalla en la sección 5. Este trabajo usa el clasificador de bosques aleatorios de scikit-learn [6].

3 Breve descripción de los algoritmos desarrollados

3.1 C4.5

El Algoritmo 1 resume la construcción del árbol. El procedimiento recibe el conjunto de casos, el conjunto de atributos disponibles $\mathcal{A}$, la profundidad d, los cuatro controles de crecimiento y devuelve un nodo. La masa de un nodo es la suma de los pesos de sus casos, de modo que los casos repartidos por valores faltantes no cuentan como filas completas. Las paradas estructurales se evalúan antes de elegir la división; la comparación con el umbral de ganancia y con la tasa mínima se aplica después, y es estricta: si el valor es menor, el crecimiento se detiene; la igualdad permite dividir.

Algoritmo 1 Construcción recursiva de un árbol C4.5

```
Calcular la masa por clase y elegir la clase mayoritaria.
si clase única, A vacío, profundidad máxima o masa mínima entonces
    devolver hoja con la clase mayoritaria.
fin si
para cada atributo a ∈ A hacer
    Evaluar sus divisiones y conservar la mejor.
fin para
Retener los candidatos con ganancia al menos promedio.
Elegir entre ellos el de mayor tasa de ganancia ajustada.
si no hay candidato o no se alcanzan los mínimos entonces
    devolver hoja con la clase mayoritaria.
fin si
Crear el nodo con el atributo, el umbral o las ramas, y las proporciones.
para cada rama hacer
    Repartir los faltantes y construir el hijo con d + 1.
fin para
devolver el nodo.
```

La tasa de ganancia puede favorecer divisiones con información de la partición muy baja, aunque aporten poco. Por eso solo compiten los atributos cuya ganancia alcanza al menos el promedio de los evaluados [4]. Los empates se resuelven por el orden de los atributos y, entre cortes numéricos del mismo atributo, por el menor umbral. Un atributo categórico se elimina del conjunto disponible después de usarse; uno numérico se conserva, porque los subconjuntos resultantes admiten nuevos cortes. En la predicción, un valor conocido sigue una sola rama; un valor faltante combina las distribuciones de todas las ramas con las proporciones aprendidas, y una categoría no vista usa las frecuencias locales del nodo

alcanzado. La clase final es la de mayor probabilidad y los empates se resuelven con la primera clase observada en el entrenamiento.

3.2 Random Forest

La implementación de Random Forest usa scikit-learn [6]. Los valores faltantes se completan antes de construir los árboles, en el mismo flujo que codifica las categorías.

4 Descripción de la solución implementada

4.1 Arquitectura y protocolo de evaluación

La solución se organiza en tres capas: preparación de los datos, modelos y evaluación. La preparación carga el conjunto de datos, deduce el tipo de cada atributo y construye una partición estratificada con el 80 % de los casos para entrenamiento y el 20 % para prueba, con una semilla fija de 42. Ambos modelos usan las mismas particiones y las mismas métricas, y ninguno accede al conjunto de prueba durante el entrenamiento. C4.5 admite valores faltantes y categorías nativas. Random Forest recibe los valores faltantes ya completados, con la mediana en los atributos numéricos y el valor más frecuente en los categóricos. Random Forest transforma las categorías en columnas binarias dentro de su flujo.

El conjunto de datos tiene 303 casos y 6 de ellos, cerca del 2 %, presentan un valor faltante: 4 en la cantidad de vasos principales y 2 en el atributo thal. Se optó por completar esos valores en lugar de descartar los casos, porque descartarlos implica perder datos que pueden ser valiosos aunque estén incompletos [5]. Esta decisión conserva los 303 casos y las mismas particiones para ambos modelos, y permite que C4.5 aplique su tratamiento propio. Los valores de reemplazo se calculan solo con el conjunto de entrenamiento, para que el conjunto de prueba no influya en la preparación.

La codificación de Random Forest se ajusta solo con el conjunto de entrenamiento y una categoría no vista se representa con ceros; C4.5 conserva una rama por categoría. Esas diferencias de preprocesamiento son una posible fuente de sesgo en la comparación y se retoman al interpretar los resultados. Ambos modelos exponen el mismo contrato de predicción: reciben una tabla de datos, devuelven una columna con la clase predicha y conservan el índice y el orden de las filas sin modificar la entrada. La validación exige las mismas columnas y tipos del entrenamiento, y los desempates del árbol son deterministas. Las probabilidades del árbol son frecuencias ponderadas y no están calibradas.

4.2 Decisiones de diseño y problemas encontrados

Los principales problemas fueron el tratamiento de atributos continuos, los valores faltantes, el control del crecimiento y la comparabilidad del preprocesamiento. La Tabla 1 resume los tratamientos adoptados o previstos. Las pruebas

de cortes, conservación de masa y repetición de particiones respaldan los tratamientos de mayor riesgo. El crecimiento se controla con cuatro parámetros independientes: el umbral de ganancia, la profundidad máxima, la masa mínima y la tasa de ganancia mínima; no se implementa pospoda.

Tabla 1. Problemas técnicos y tratamientos adoptados o previstos.

Problema	Tratamiento
Atributos continuos	Cortes binarios en valores observados; sin grilla global
Valores faltantes	C4.5 los reparte con peso proporcional; Random Forest los recibe completados
Crecimiento	Prepoda con cuatro controles; sin pospoda
Preprocesamiento	Categorías nativas en C4.5; codificación binaria en Random Forest

5 Simulaciones y resultados

5.1 Configuración experimental

Se usaron los 303 casos del conjunto de datos, con la clase agrupada en ausencia y presencia de enfermedad. El experimento se repitió con 10 particiones estratificadas distintas, cada una con 242 casos de entrenamiento y 61 de prueba. Ambos modelos recibieron las mismas particiones. Con 61 casos, un acierto mueve la tasa cerca de 1,6 puntos porcentuales, por lo que una sola partición resulta poco confiable; por ese motivo se informan la media y el desvío de las repeticiones.

Las métricas son la tasa de acierto y las versiones macro de precisión, exhaustividad y F1, que promedian ambas clases con igual peso. La variación de parámetros se usa para describir tendencias y no para seleccionar una configuración final. Por eso los valores por defecto son los que sostienen la comparación principal.

5.2 Comparación con los valores por defecto

La Tabla 2 resume la comparación. Random Forest obtiene una tasa de acierto de 0,811 y C4.5, de 0,718. Ambos alcanzan 1,000 en entrenamiento. El árbol sin límites tiene en promedio 87 nodos, 46 hojas y profundidad 11: crece hasta separar casi cada caso de entrenamiento. Esta diferencia entre entrenamiento y prueba se denomina sobreajuste. En Random Forest también aparece la tasa perfecta en entrenamiento, pero el voto de muchos árboles compensa los errores individuales y conserva mejor rendimiento en prueba [1].

Los tiempos no comparan los algoritmos en igualdad de condiciones. C4.5 está escrito en Python para mostrar el procedimiento, mientras que scikit-learn

Tabla 2. Resultados sobre el conjunto de prueba, con media y desvío de 10 particiones.

Métrica	C4.5	Random Forest
Tasa de acierto	$0{,}718 \pm 0{,}043$	$0{,}811 \pm 0{,}047$
Precisión	$0{,}723 \pm 0{,}040$	$0{,}816 \pm 0{,}048$
Exhaustividad	$0{,}719 \pm 0{,}040$	$0{,}808 \pm 0{,}049$
F1	$0{,}716 \pm 0{,}042$	$0{,}808 \pm 0{,}049$
Tasa de acierto en entrenamiento	1,000	1,000
Tiempo de entrenamiento (ms)	989	88
Tiempo de predicción (ms)	2,4	8,4

usa código compilado. Aun así, predecir con un solo árbol resulta más rápido que consultar 100 árboles.

La Figura 1 muestra las matrices de confusión. Cada fila es la clase real y cada columna, la clase predicha; la diagonal contiene los aciertos. Cada matriz suma 610 predicciones, 61 casos por 10 particiones. La celda de presencia real y predicción de ausencia reúne los falsos negativos, el error de mayor costo en este problema, porque deja a un enfermo sin detectar. C4.5 comete 75 de esos errores sobre 280 casos con enfermedad y Random Forest, 64. Random Forest también genera menos falsas alarmas: 51 contra 97.

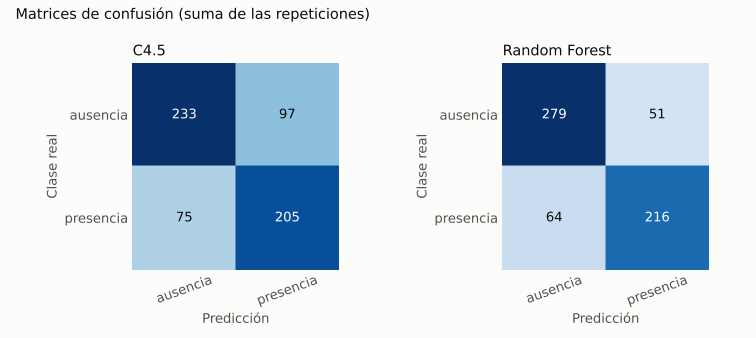


Figura 1. Matrices de confusión de ambos modelos, sumadas sobre las 10 particiones.

5.3 Efecto de cada parámetro

Se varió un parámetro por vez y los demás quedaron en su valor por defecto. Las Figuras 2 y 3 se leen de la siguiente manera. La línea continua es la tasa de acierto en prueba y la discontinua, en entrenamiento. La banda sombreada abarca un desvío hacia cada lado de la media. Si las dos curvas son bajas, el modelo es demasiado simple y hay subajuste. Si la de entrenamiento sube y la de prueba no acompaña, hay sobreajuste. Si las bandas de dos valores se

superponen, la diferencia entre ellos no alcanza para afirmar que uno es mejor. La Tabla 3 reúne todas las variaciones.

Tabla 3. Tasa de acierto media al variar cada parámetro, en prueba y en entrenamiento.

Modelo y parámetro	Valor	Prueba	Entrenamiento
C4.5: profundidad máxima	1 / 4 / 10	0,703 / 0,757 / 0,720	0,753 / 0,876 / 0,993
C4.5: masa mínima para dividir	2 / 10 / 40	0,718 / 0,752 / 0,739	1,000 / 0,925 / 0,812
C4.5: ganancia mínima (bits)	0 / 0,05 / 0,1	0,718 / 0,731 / 0,741	1,000 / 0,982 / 0,876
C4.5: tasa de ganancia mínima	0 / 0,1 / 0,2	0,718 / 0,744 / 0,554	1,000 / 0,937 / 0,573
Random Forest: cantidad de árboles	10 / 50 / 200	0,797 / 0,811 / 0,808	0,990 / 1,000 / 1,000
Random Forest: profundidad máxima	1 / 3 / 10	0,816 / 0,825 / 0,807	0,845 / 0,878 / 1,000

En C4.5, la profundidad máxima controla cuántas preguntas puede encadenar el árbol. Con un nivel la tasa en prueba es 0,703; llega a 0,757 con cuatro niveles y luego baja a 0,720, mientras que la de entrenamiento sigue subiendo hasta 0,993. A partir de ese punto el árbol aprende detalles del entrenamiento que no se repiten en casos nuevos.

La masa mínima para dividir impide partir nodos con pocos casos. Al subirla de 2 a 10, la tasa en entrenamiento baja de 1,000 a 0,925 y la de prueba sube de 0,718 a 0,752, de modo que el árbol pierde reglas apoyadas en muy pocos casos. Con 40 la prueba baja a 0,739. La ganancia mínima no cambia el árbol hasta 0,02 bits y con 0,1 bits lleva la prueba a 0,741. La tasa de ganancia mínima tampoco lo cambia hasta 0,05 y mejora con 0,1, pero con 0,2 lleva la prueba a 0,554, un valor cercano a predecir siempre la clase más frecuente, que representa el 54 % de los casos: el árbol queda en subajuste.

El filtro de ganancia también se evaluó por separado. Sin él, y eligiendo los umbrales por tasa de ganancia, el árbol sin límites obtiene 0,708 en lugar de 0,718, una diferencia menor que el desvío. Con un solo nivel, en cambio, la tasa baja de 0,703 a 0,646: el primer corte separa muy pocos casos y aporta poca información.

Random Forest resulta mucho menos sensible. Con 10 árboles alcanza 0,797 y desde 50 se estabiliza cerca de 0,81; agregar más árboles encarece el cálculo sin una mejora visible. Limitar la profundidad apenas cambia el resultado: va de 0,807 a 0,825 y sus bandas se superponen, incluso con árboles de un solo nivel. Las opciones restantes tampoco muestran diferencias mayores que el desvío: entre raíz cuadrada, logaritmo y todos los atributos por división la tasa va de 0,807 a 0,812; con Gini es 0,812 y con entropía, 0,805; con muestreo con reemplazo es 0,812 y sin él, 0,808.

6 Conclusiones

Sobre este conjunto de datos, Random Forest obtuvo mejor rendimiento que el árbol C4.5 propio: 0,811 contra 0,718 de tasa de acierto con los valores por

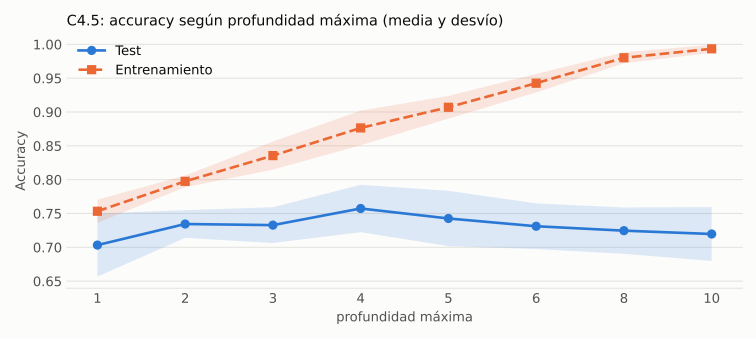


Figura 2. C4.5: efecto de la profundidad máxima.

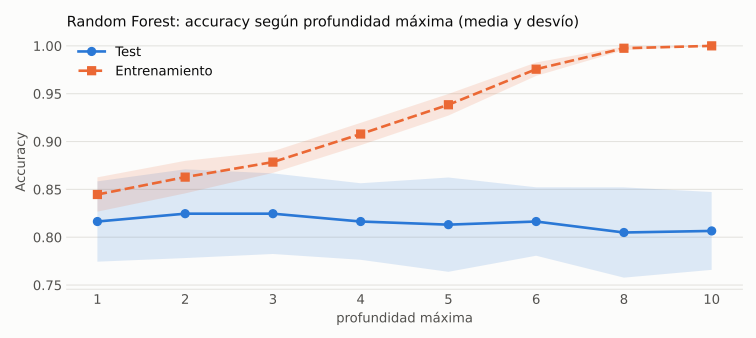


Figura 3. Random Forest: efecto de la profundidad máxima.

defecto, y menos falsos negativos. Limitar el crecimiento del árbol redujo esa distancia y llevó su tasa hasta 0,757, sin alcanzar al ensamble.

La diferencia de estabilidad fue tan importante como la de rendimiento. C4.5 cambió de manera marcada con sus parámetros, por lo que requiere ajuste. Random Forest mantuvo resultados similares en casi todas las configuraciones probadas. A cambio, C4.5 produce un modelo legible, en el que cada predicción se explica con un camino de reglas, y esa propiedad se pierde al combinar 100 árboles.

Los resultados tienen límites. Se usó un solo conjunto de datos, de 303 casos, y varias diferencias quedan dentro del desvío. La implementación no incluye la pospoda del C4.5 original, que podría reducir el sobreajuste sin fijar un límite de antemano. Queda como trabajo futuro incorporarla y repetir la comparación sobre otros conjuntos de datos.

Referencias

1. Breiman, L.: Random forests. Machine Learning **45**(1), 5–32 (2001). https://doi.org/10.1023/A:1010933404324

2. Janosi, A., Steinbrunn, W., Pfisterer, M., Detrano, R.: Heart disease. UCI Machine Learning Repository (1989). https://doi.org/10.24432/C52P4X
3. Liu, B.: Web Data Mining: Exploring Hyperlinks, Contents, and Usage Data. Data-Centric Systems and Applications, Springer, Berlin, Heidelberg, 2 edn. (2011). https://doi.org/10.1007/978-3-642-19460-3
4. Quinlan, J.R.: C4.5: Programs for Machine Learning. Morgan Kaufmann, San Mateo, CA (1993)
5. scikit-learn developers: Imputation of missing values. `https://scikit-learn.org/stable/modules/impute.html`, accessed 2026-11-06
6. scikit-learn developers: Random forests. `https://scikit-learn.org/stable/modules/ensemble.html#forest`, accessed 2026-10-15